# Team Deliverable 3 Report

Brian Tramuel, Will Armentrout, Sam Suther, Isaac Vunzi, and Sana Ullah

Library Management System

## A. Phase 1 Requirements

The scope of Phase 1 is unchanged from Team Deliverable 2. Phase 1 targets the core authentication and catalog-browsing functionality required to make the system usable for the first time. The requirements below are the same as those specified in Deliverable 2, but have been expanded with detailed acceptance criteria and implementation notes.

**Phase Overview**

| Phase | Focus | Included Requirements |
|---|---|---|
| Phase 1 | MVP Auth and Catalog | FR-1 to FR-8, NFR-1 to NFR-4, NFR-9, NFR-10 |
| Phase 2 | Core Security, Borrow and Return | FR-9 to FR-16, NFR-5 to NFR-7 |
| Phase 3 | Admin Tools & Polish | FR-17 to FR-23, NFR-8, NFR-11, NFR-12 |

## Functional Requirements

| ID | Name | Detailed Description |
|---|---|---|
| FR-1 | User Registration | Users provide a full name, a unique email, a unique username, and a password meeting complexity rules. Duplicate emails or usernames return a specific error. On success, the account is saved, and the user is redirected to the login page. |
| FR-2 | Login / Logout | Registered users authenticate with username and password. A JWT is issued on success and stored client-side. Logout invalidates the token server-side. Protected pages redirect unauthenticated visitors to the login |
| FR-3 | Role-Based Access | Every account carries a role: User or Admin. The default at registration is User. Admin role is granted only from a privileged DB operation or by another admin. Routes return HTTP 403 when accessed without the required role. |
| FR-4 | Password Requirements | Passwords must be at least 8 characters and include one uppercase letter, one lowercase letter, one digit, and one special character. The UI displays a real-time strength indicator and blocks submission until all criteria are met |
| FR-5 | Browse Catalog | Any visitor can view a paginated list of active books. Each card shows title, author, genre, and an availability badge. |

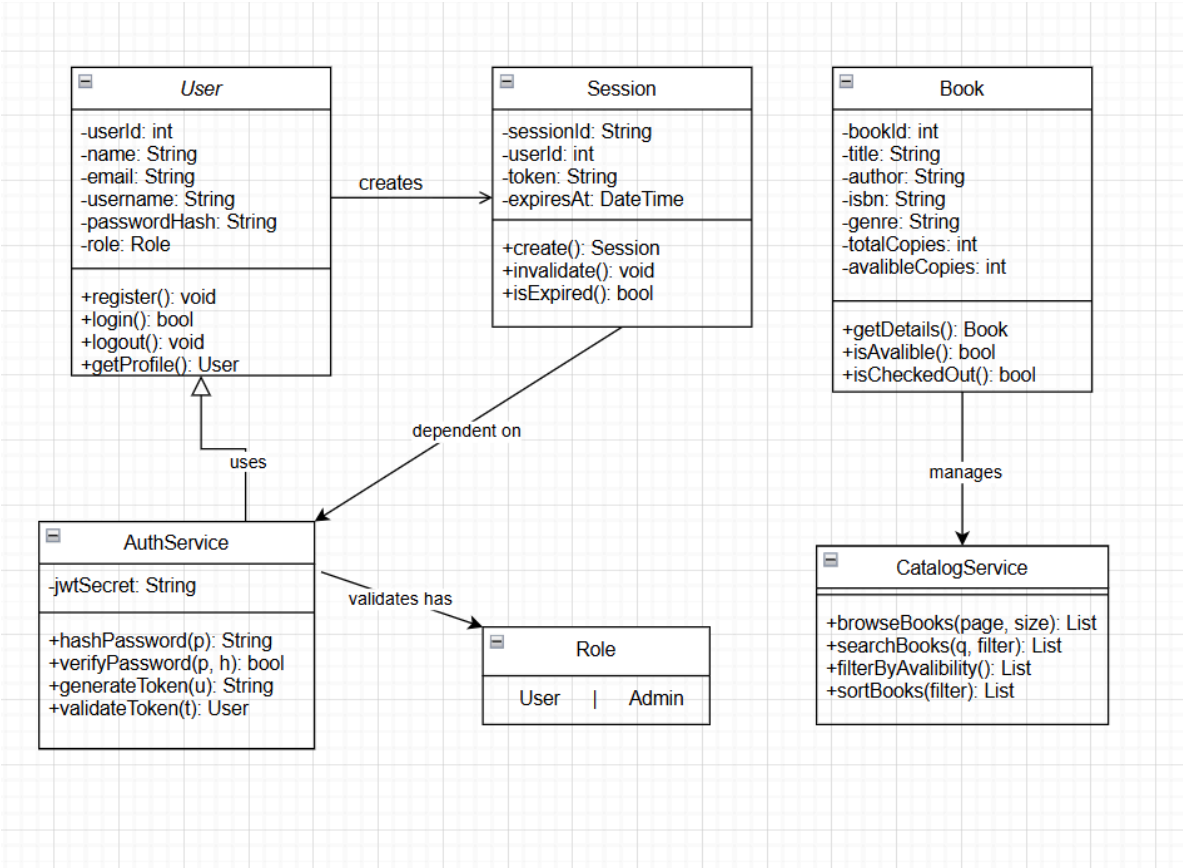
| | | |
|---|---|---|
| | | Default sort is alphabetical by title. |
| FR-6 | Search By Title / Author | A search bar accepts free-text queries. The backend runs a case-insensitive search on the title and author columns. Results appear within 2 seconds; an empty-state message is shown when no books match |
| FR-7 | Filter & Sort | Filters: availability and genre. Sort options: Title A-Z, Title Z-A, Author A-Z, Newest Added. Selections persist across pagination without resetting the page counter. |
| FR-8 | Book Detail View | Clicking a book opens a detail page showing title, author, ISBN, genre, total copies, available copies, and status. Authenticated users see a Borrow button. Guests see a Login to Borrow prompt |

## Non-Functional Requirements

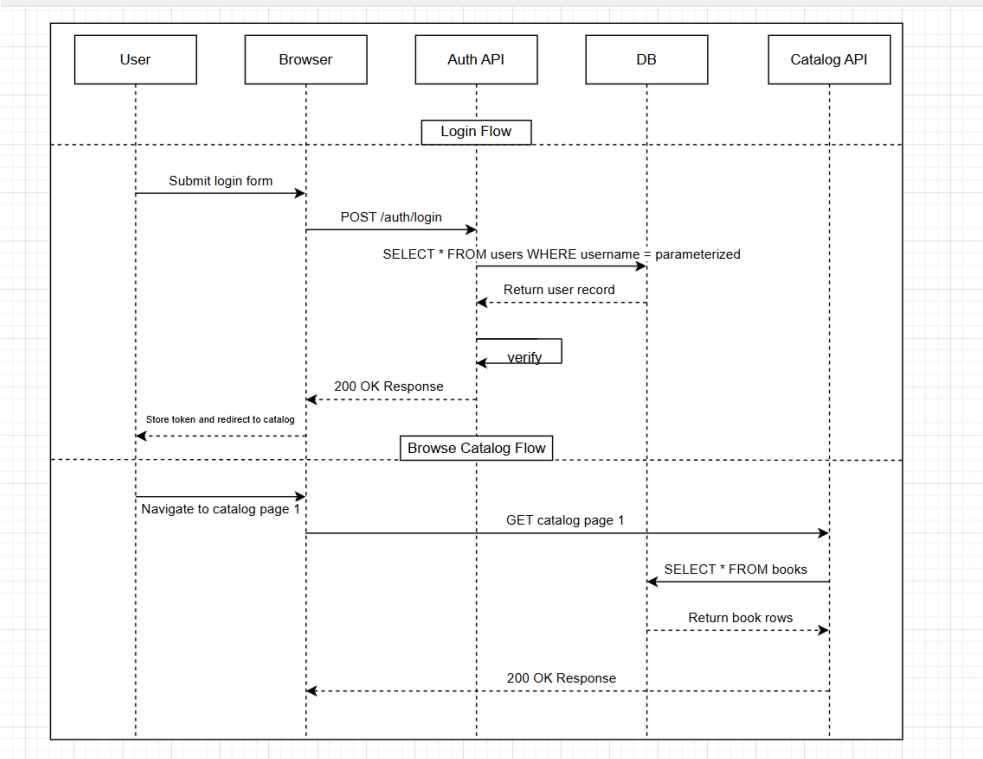
| ID | Category | Requirement and Detail |
|---|---|---|
| NFR-1 | Security | Passwords are hashed with bcrypt or argon2id before storage. Plain-text passwords are never written to the database or logs. |
| NFR-2 | Security | Admin-only API routes are protected by a server-side role check on every request. Client-side hiding of UI elements alone is not sufficient protection. |
| NFR-3 | Security | JWTs expire after 30 minutes of inactivity. Refresh tokens rotate on each use. HTTPS is enforced in production; HTTP-only cookies are used where applicable. |
| NFR-4 | Security | All user inputs are sanitized via parameterized SQL statements and output-encoded to prevent XSS. The OWASP Top 10 is used as an implementation checklist. |
| NFR-9 | Usability | A first-time user can complete registration and view the catalog within 3 minutes with no prior training. Verified by a structured usability walkthrough before release. |
| NFR-10 | Accessability | UI follows WCAG 2.1 AA: all form fields have labels, and interactive elements are keyboard-navigable. |

## B. UML Design For Phase 1

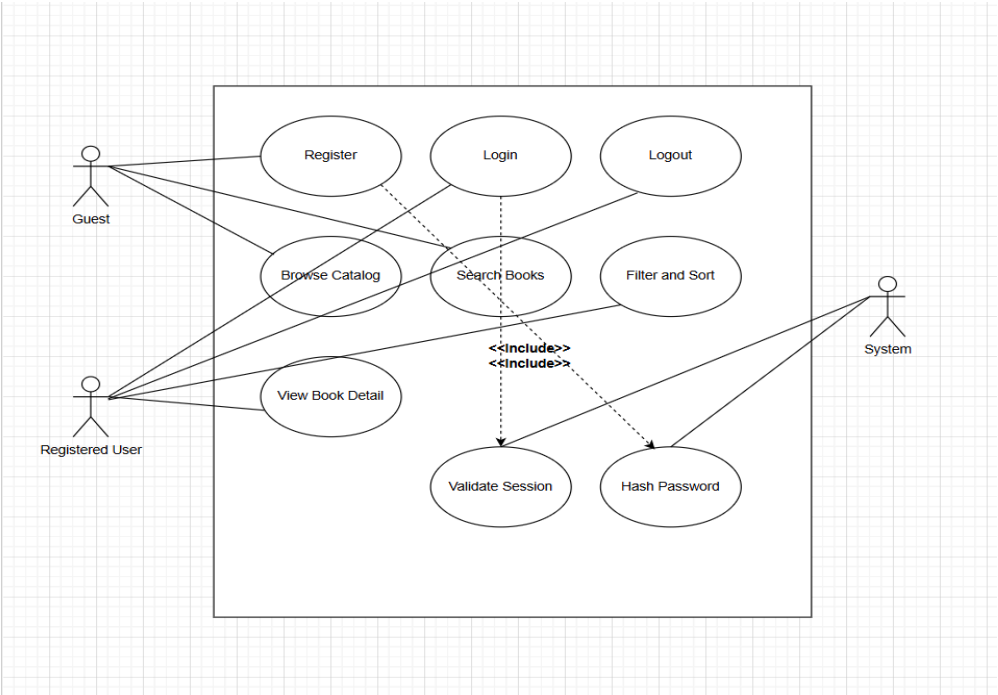
## Class Diagram




## Sequence Diagram



## Use-Case Diagram




## C. User Manuel

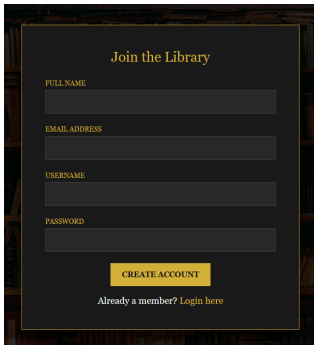
This manual explains how to access and use the Library Management System as it currently exists after Phase 1. The application is a web-based system with a Node.js/Express backend, a SQL database, and an HTML/CSS/JavaScript frontend styled with a dark theme.

### Accessing the Application

Once the server is running, open your browser and go to *http://localhost:3000*. You will see the Library home page with a centered layout, a Dark Academia-themed design, and navigation links for Login, Register, and Search in the header.

### Registering a New Account

1. Click **Register** in the navigation bar. The registration page will load.
2. Fill in your **Full Name, Email Address, Username, and Password**.
3. Your password must be at least **8** characters and include one uppercase letter, one lowercase letter, one digit, and one special character.
4. Click **Create Account**. On success, you are redirected to the login page. If the username or email is already taken, an error message will appear below the field.



### Logging In and Out

Click Login in the navigation bar. Enter your username and password, then click Sign In. After a successful login, your session is stored, and you are redirected to the catalog. To log out, click the Logout link in the navigation bar; your session will be cleared immediately.

### Searching and Browsing the Catalog

The Search page is accessible from the navigation bar at any time, whether or not you are logged in.

- **Search bar**: Type a book title or author name and press Enter or click Search. Results update on the page without a full reload.
- **Browse all**: Leave the search bar empty and submit to see all available books.
- **Book cards**: Each result card shows the title, author, genre, and availability status.
- **Book Detai**l: Click a book title or card to open its full detail page, which lists ISBN, description, publication year, total copies, and available copies.

## D. Compile/Run Instructions

The project uses Node.js and Express for the backend, PostgreSQL for the database, and plain HTML/CSS/JavaScript for the frontend. All source files are on the main branch of the GitHub repository.

**Must Have:**

- **Node.js**
- **Git**
- **PostgreSQL**

### Step 1 - Clone Git

git clone https://github.com/ssuther6-ops/ITIS3300-Panthers.git cd ITIS3300-Panthers

### Step 2 - Install Dependencies

The project dependencies are declared in package.json.
Install them with: **npm install**

### Step 3 - Create and Configure the Database

Open a psql shell and create the database:
**CREATE DATABASE library_db;**

Then run the schema script to create the tables:
**psql -d library_db -f src/database/schema.sql**

### Step 4 - Set Environment Variables

Create a .env file in the project root and set the following:

| Variable | Example Value |
|---|---|

| | |
|---|---|
| DB_Host | localhost |
| DB_Port | 5432 |
| DB_Name | library_db |
| DB_User | postgres |
| DB_Password | Passw0rd |
| JWT_Secret | panthers3300 |
| PORT | 3000 |

### Step 5 - Start the Server

**node src/backend/server.js**

You should see: Server running on port 3000. Open *http://localhost:3000* in your browser to use the application.

### Running Test

| **EndPoint** | **Method** |
|---|---|
| /api/auth/register | POST |
| /api/auth/login | POST |
| /api/books | GET |
| /api/books?search=query | GET |
| /api/users | GET |

## E. Reflection

**What has been accomplished:**

Phase 1 is complete, and the team built a working full-stack web application from the ground up. The backend Express server is live with routing and middleware in place. The database connection pool and schema are initialised via db.js. User registration and login are functional, with bcrypt password hashing and JWT-based authentication implemented in userController.js. Book catalog routes are documented in bookController.js, and the borrow/return API skeleton has been created ahead of Phase 2. On the frontend, the login, register, and search pages are complete with a consistent Dark Academia Library theme. Admin management routes for both books and users have been added and documented.

**What Went Well:**

- **Steady commitment**: The team maintained consistent daily commits across all members, with clear and descriptive commit messages that made progress easy to track.
- **Clear separation of concerns**: The decision to separate controllers, routes, and the database layer from the start made each file focused and straightforward to review.
- **Frontend progress**: The login, register, and search pages were completed with a polished, cohesive theme, giving the project a professional appearance early.
- **Early Phase 2 progress**: Creating borrow.js and borrowController.js during Phase 1 means the team has a head start on the next deliverable.

### Areas Of Improvement:

- **Language decision:** The team spent time early on deciding between Node.js and Python before settling on Node.js/Express. Locking in the desired language earlier would've freed up more time for implementation.
- **Frontend and backend integration:** The frontend HTML pages and the Express API were built somewhat in parallel.
- **Git workflow:** Most commits went directly to the main branch. Using feature branches and pull requests would allow for code review before merging and reduce the risk of breaking working code.

## F. Contribution Table

| Member Name | Contribution Description | Overall Contribution % | Note |
|---|---|---|---|
| **Will** | Filled out phase requirements, overview, functional requirements, non-functional requirements, class diagram, sequence diagram, use-case diagram, reflection, user manual, compile instructions, add authController, authRoutes, and auth code. | **20%** | |
| **Brian** | Built and set up the entire backend, including the server, database connection, and also the controllers and routes for books, users, and borrowing | **20%** | |
| **Isaac** | Finished the frontend skeletons for the Login, Register, and Search pages. Everything is styled with a Library theme and linked together | **20%** | |
| **Sam** | Implemented the frontend password strength indicator for the registration page, including a real-time strength bar, requirement checklist with checkmark feedback, color-coded strength levels weak, medium, and strong, and a submit button blocking until all password criteria are met. | **20%** | |

| | | | |
|---|---|---|---|
| **Sana** | Implemented authentication and security features for the Library Management System, including registration and login logic, password strength validation, bcrypt password hashing, JWT-based authentication, protected routes, admin role checks, and SQL user schema support. | **20%** | |